

UNIVERSIDAD NACIONAL DE INGENIERÍA

FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA



INFORME: PROGRAMACION DINÁMICA

GRUPO N°3

INTEGRANTES:

- MARCAS UNOCC, LUIS ANGEL (20264013D)
- ZEVALLOS TORRES, RAUL HENRY (20171258G)

CÓDIGO DEL CURSO: BMA09 (ALGORITMO Y ESTRUCTURA DE DATOS 1)

SECCIÓN: T

DOCENTE: SELIS VASQUEZ, LUIS ANTONO

2026-2

1. Concepto de Programación Dinámica

Definición:

La Programación Dinámica (PD) es una estrategia de diseño de algoritmos que resuelve un problema complejo **descomponiéndolo en subproblemas más simples**, resolviendo cada subproblema **una sola vez** y **almacenando sus resultados** para reutilizarlos cuando sea necesario.

Características principales:

- Se usa cuando el problema tiene **subproblemas superpuestos** (se repiten).
- Requiere **subestructura óptima** (la solución óptima del problema se construye a partir de soluciones óptimas de subproblemas).
- Evita recalcular lo ya resuelto mediante **memoización** o **tabulación**.
- Generalmente transforma un algoritmo exponencial en uno polinómico.

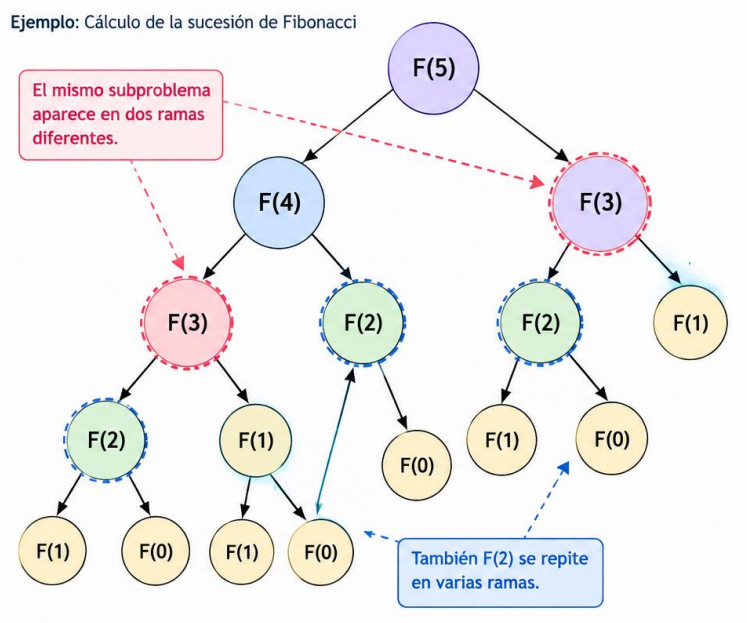
Analogía:

Imagina que estás calculando cuántos caminos hay para subir una escalera de 10 peldaños, donde puedes subir 1 o 2 peldaños a la vez. Si ya calculaste cuántos caminos hay para llegar al peldaño 5, no necesitas volver a calcularlo cada vez que lo necesites; lo guardas y lo reutilizas.

2. Subproblemas superpuestos

Un problema tiene **subproblemas superpuestos** cuando al descomponerlo recursivamente, **el mismo subproblema aparece múltiples veces**.

Ejemplo clásico: Fibonacci.



Si calculas $F(5)$ recursivamente, calcularás $F(3)$ varias veces. Eso es superposición.

Sin Programación Dinámica: complejidad exponencial $O(2^n)$.

Con Programación Dinámica: complejidad lineal $O(n)$.

3. Subestructura óptima

Un problema tiene **subestructura óptima** si la solución óptima del problema se puede construir a partir de las soluciones óptimas de sus subproblemas.

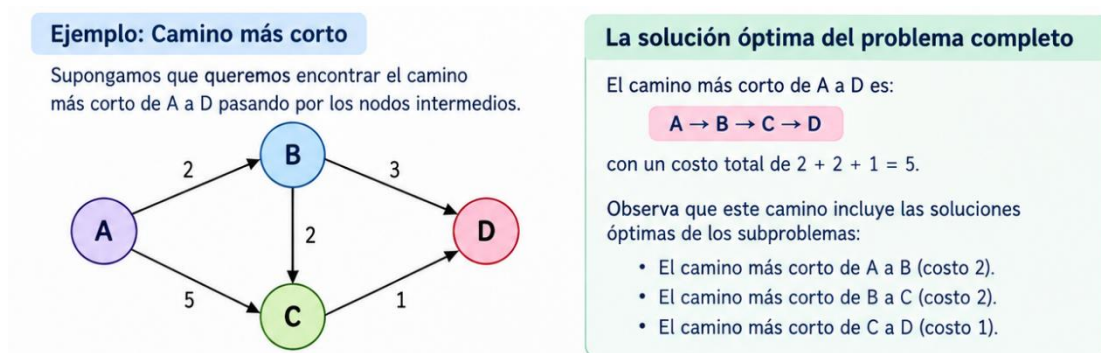
Ejemplo: Camino más corto en un grafo (Dijkstra).

Si el camino más corto de A a C pasa por B, entonces el tramo $A \rightarrow B$ debe ser el más corto posible, y $B \rightarrow C$ también.

Importancia:

Sin esta propiedad, no podemos garantizar que combinar soluciones óptimas de subproblemas nos dé la solución óptima global.

De forma ilustrada:



Nota: el problema inicial sería el camino con menor coste(recursos).

4. Memoización

La memoización es una forma de caching (técnica que guarda copias temporales de datos en una capa de almacenamiento de alta velocidad para acelerar futuras solicitudes de esa misma información) aplicada a funciones. Cuando una función se llama con ciertos argumentos:

Se verifica si el resultado para esos argumentos ya está guardado.

Si está; se devuelve directamente (evitando el cálculo).

Si no está; se calcula, se guarda y se devuelve.

Características clave

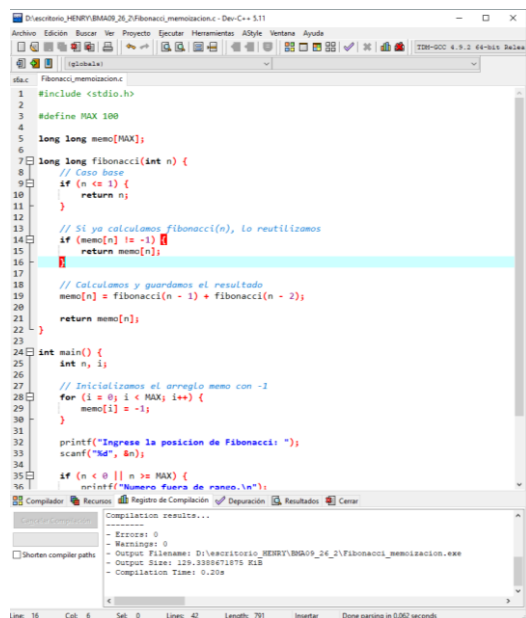
Requiere funciones puras: mismas entradas $\rightarrow$ mismas salidas (sin efectos secundarios).

No confundir con memorización: es un término técnico, no tiene que ver con la memoria humana.

Se usa mucho en programación dinámica (Top-Down).

Complejidad temporal se reduce a cambio de usar más memoria (trade-off tiempo-espacio).

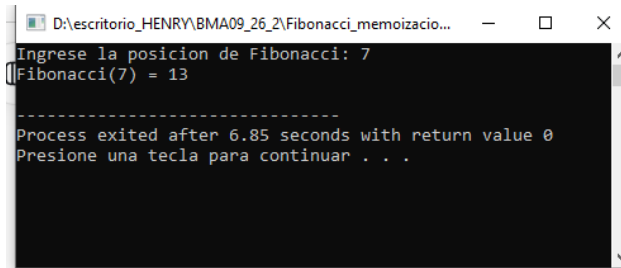
Ejemplo Fibonacci con memoización:



```
1 #include <stdio.h>
2
3 #define MAX 100
4
5 long long memo[MAX];
6
7 long long fibonacci(int n) {
8     // Caso base
9     if (n <= 1) {
10         return n;
11     }
12
13     // Si ya calculamos fibonacci(n), lo reutilizamos
14     if (memo[n] != -1) {
15         return memo[n];
16     }
17
18     // Calculamos y guardamos el resultado
19     memo[n] = fibonacci(n - 1) + fibonacci(n - 2);
20
21     return memo[n];
22 }
23
24 int main() {
25     int n, i;
26
27     // Inicializamos el arreglo memo con -1
28     for (i = 0; i < MAX; i++) {
29         memo[i] = -1;
30     }
31
32     printf("Ingrese la posición de Fibonacci: ");
33     scanf("%d", &n);
34
35     if (n < 0 || n >= MAX) {
36         printf("Número fuera de rango.\n");
37     }
38
39     printf("Fibonacci(%d) = %lld\n", n, fibonacci(n));
40 }
```

Compilation results:

```
-----
- Errors: 0
- Warnings: 0
- Output File(s): D:\escritorio_HENRY\BMA09_26\Fibonacci_memoizacio...
- Output Size: 129,338,671,675 B
- Compilation Time: 0.20s
```



```
D:\escritorio_HENRY\BMA09_26\Fibonacci_memoizacio...
Ingrese la posición de Fibonacci: 7
Fibonacci(7) = 13

-----
Process exited after 6.85 seconds with return value 0
Presione una tecla para continuar . . .
```

Ventaja: Sin recursión, más eficiente en memoria y tiempo.

Desventaja: A veces calculas subproblemas que no necesitas.

5. Tabulación

La tabulación (también llamada *método bottom-up* o "de abajo hacia arriba") es una técnica dentro de la Programación Dinámica que consiste en: **Resolver un problema guardando los resultados de los subproblemas más pequeños en una tabla, e ir construyendo la solución del problema grande a partir de ellos.**

En lugar de resolver recursivamente el mismo subproblema muchas veces, **calculamos cada subproblema UNA sola vez y lo almacenamos** en una tabla (usualmente un arreglo o matriz).

La idea central

Imagina que quieres calcular **Fibonacci (5)**.

Sin programación dinámica (recursión pura):

$F(5) = F(4) + F(3)$
 $F(4) = F(3) + F(2) \leftarrow F(3)$ se calcula otra vez
 $F(3) = F(2) + F(1) \leftarrow F(2)$ se calcula otra vez...

Se repiten cálculos **miles de veces**. Es lento y desperdicia tiempo.

Con tabulación:

$F(0) = 0$
 $F(1) = 1$
 $F(2) = F(1) + F(0) = 1$
 $F(3) = F(2) + F(1) = 2$
 $F(4) = F(3) + F(2) = 3$
 $F(5) = F(4) + F(3) = 5$

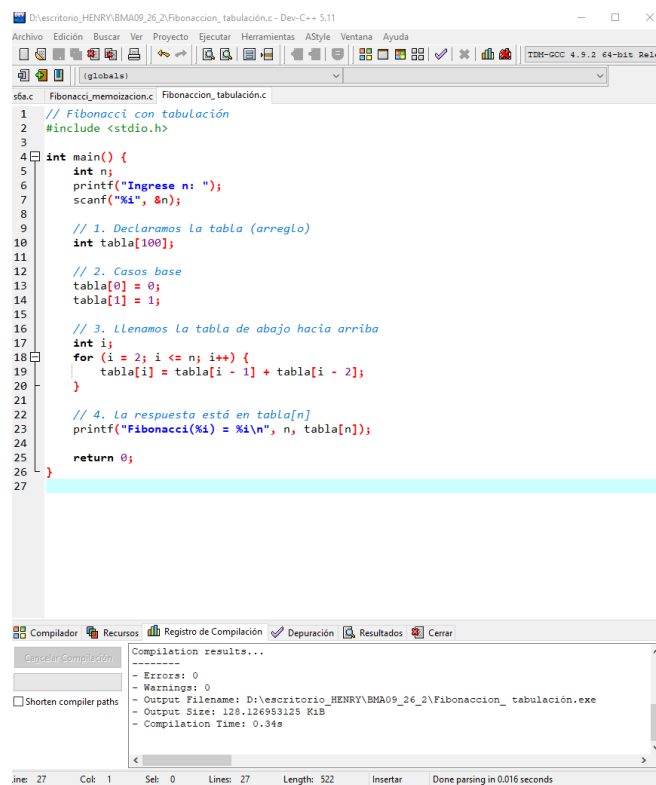
Cada valor se calcula **una sola vez** y se guarda en una tabla (tabla [0..5]).

¿Cómo se construye una solución con tabulación?

Pasos generales

1. **Identificar el estado:** ¿qué información necesito guardar en cada celda de la tabla?
2. **Definir la tabla:** tamaño y tipo (arreglo 1D, 2D, etc.).
3. **Caso base:** llenar las celdas más pequeñas (las que no dependen de nadie).
4. **Relación de recurrencia:** fórmula que expresa un estado en función de estados anteriores.
5. **Orden de llenado:** recorrer la tabla desde los casos base hacia arriba.
6. **Respuesta:** normalmente está en la última celda calculada.

Ejemplo Fibonacci con tabulación:

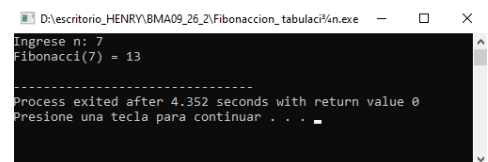


```
1 // Fibonacci con tabulación
2 #include <stdio.h>
3
4 int main() {
5     int n;
6     printf("Ingrese n: ");
7     scanf("%i", &n);
8
9     // 1. Declaramos la tabla (arreglo)
10    int tabla[100];
11
12    // 2. Casos base
13    tabla[0] = 0;
14    tabla[1] = 1;
15
16    // 3. Llenamos la tabla de abajo hacia arriba
17    int i;
18    for (i = 2; i <= n; i++) {
19        tabla[i] = tabla[i - 1] + tabla[i - 2];
20    }
21
22    // 4. La respuesta está en tabla[n]
23    printf("Fibonacci(%i) = %i\n", n, tabla[n]);
24
25    return 0;
26 }
27
```

Compilation results...

```
- Errors: 0
- Warnings: 0
- Output Filename: D:\escritorio_HENRY\BMA09_26_2\Fibonacci_tabulación.exe
- Output Size: 128.126953125 KiB
- Compilation Time: 0.34s
```

Line: 27 Col: 1 Sel: 0 Lines: 27 Length: 522 Insertar Done parsing in 0.016 seconds



```
Ingrese n: 7
Fibonacci(7) = 13

.....
Process exited after 4.352 seconds with return value 0
Presione una tecla para continuar . . .
```

Tabulación vs. Memoización (para que no las confundas)

Aspecto	Tabulación (bottom-up)	Memoización (top-down)
Dirección	De lo pequeño a lo grande	De lo grande a lo pequeño
Herramienta	Bucles (for)	Recursión + caché
Riesgo de stack overflow	No	Sí, si hay mucha recursión
Cálculo de subproblemas innecesarios	Puede calcular algunos que no se usan	Solo calcula los que se necesitan
Fácil de depurar	Sí	A veces más difícil

Ambas **guardan resultados**; la diferencia es **cómo se recorre el espacio de estados**.

6. Estados y transiciones

6.1. ¿Qué es un ESTADO?

Un estado es una situación o "foto" parcial del problema que necesitas resolver.

Dicho de otra forma: un estado responde a la pregunta:

"¿Qué información necesito saber para resolver este subproblema?"

Cada estado se representa normalmente con:

- Uno o varios índices (parámetros).
- Un valor almacenado (la respuesta a ese subproblema).

Ejemplo clásico: Fibonacci

Quieres calcular `fib(n)`. El estado sería:

Estado = `fib(i)` → "el *i*-ésimo número de Fibonacci"

Aquí el estado se identifica con un solo parámetro: *i*.

En C lo guardarías en un arreglo:

Código en C:

```
int dp[100]; // dp[i] = estado i → valor de fib(i)
```

6.2. ¿Qué es una TRANSICIÓN?

Una **transición** es la **regla** que conecta un estado con otro.

Es la fórmula que dice: "***para calcular este estado, uso estos otros estados ya calculados***".

También se le llama **recurrencia** o **ecuación de transición**.

Siguiendo Fibonacci:

$$\text{fib}(0) = 0$$

$$\text{fib}(1) = 1$$

$$\text{fib}(i) = \text{fib}(i-1) + \text{fib}(i-2) \quad \leftarrow \text{ESTA es la transición}$$

En código C:

```
dp[0] = 0;
dp[1] = 1;
for (int i = 2; i <= n; i++) {
    dp[i] = dp[i-1] + dp[i-2]; // transición
}
```

- $\text{dp}[i]$ es el **estado**.
- $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$ es la **transición**.

Otro ejemplo: subir escaleras

Problema: puedes subir 1 o 2 escalones por paso. ¿Cuántas formas hay de llegar al escalón n ?

- **Estado:** $\text{dp}[i]$ = formas de llegar al escalón i .
- **Transición:** $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$
(porque al escalón i solo puedes llegar desde $i-1$ o desde $i-2$).
- **Casos base:** $\text{dp}[0] = 1$, $\text{dp}[1] = 1$.

En código C:

```
int dp[n+1];
dp[0] = 1;
dp[1] = 1;
for (int i = 2; i <= n; i++)
    dp[i] = dp[i-1] + dp[i-2];
```

Ejemplo con DOS parámetros: Mochila 0/1

Tienes n objetos con peso $p[i]$ y valor $v[i]$, y una mochila con capacidad C . Maximiza el valor.

Aquí el estado necesita **dos datos**:

Estado = $dp[i][c]$

= "el mejor valor usando los primeros i objetos con capacidad c ".

- **Transición:** para cada objeto decides **tomarlo o no tomarlo**:

```
dp[i][c] = max( dp[i-1][c],           // no lo tomo
               dp[i-1][c - p[i]] + v[i] ) // sí lo tomo (si cabe)
```

En código C:

```
for (int i = 1; i <= n; i++) {
    for (int c = 0; c <= C; c++) {
        dp[i][c] = dp[i-1][c];           // no tomarlo
        if (p[i] <= c)
            dp[i][c] = max(dp[i][c],
                           dp[i-1][c - p[i]] + v[i]); // tomarlo
    }
}
```

Fíjate cómo cada celda $dp[i][c]$ (estado) se calcula **a partir de estados anteriores** mediante la transición.

Resumen mental

Concepto	¿Qué es?	En Fibonacci
Estado	Subproblema identificado por parámetros	$dp[i]$
Transición	Regla que relaciona estados ya calculados	$dp[i] = dp[i-1] + dp[i-2]$
Caso base	Estados que se conocen directamente	$dp[0] = 0, dp[1] = 1$
Orden	Cómo recorrer los bucles para que la transición funcione	de $i=2$ a n
Respuesta	Estado final que buscas	$dp[n]$

Pasos para resolver un problema con PD

1. **Definir el estado:** ¿qué parámetros describen un subproblema? ($dp[i]$, $dp[i][c]$, etc.)
2. **Escribir la transición:** ¿de qué estados anteriores depende?
3. **Definir los casos base:** los valores que ya sabes.
4. **Elegir el orden de cálculo:** bucles que respeten las dependencias.
5. **Identificar la respuesta:** ¿qué celda del arreglo devuelves?

Clave: Definir bien los estados y transiciones es el 80% del trabajo en PD.

7. Enfoque Top-Down

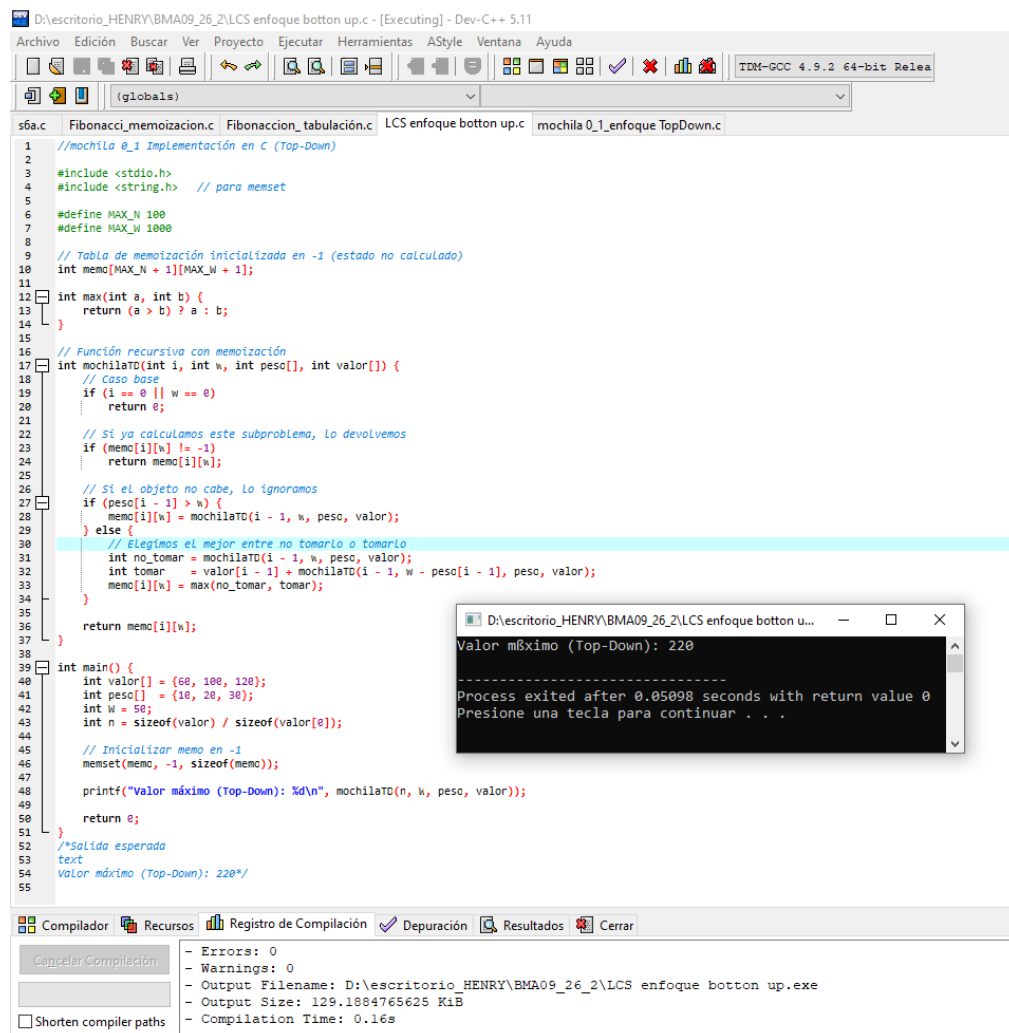
- Se parte del problema original y se resuelve recursivamente.
- Se usa **memoización** para evitar recalcular.
- Es natural si piensas en la recurrencia.

Ventaja: Solo resuelve los subproblemas necesarios.

Desventaja: Puede haber desbordamiento de pila en problemas grandes.

Ejemplo: Mochila 0/1 (Memoización)

Dado un conjunto de objetos con peso y valor, y una mochila de capacidad W, maximizar el valor sin exceder el peso.



```
1 //mochila_0_1 Implementación en C (Top-Down)
2
3 #include <stdio.h>
4 #include <string.h> // para memset
5
6 #define MAX_N 100
7 #define MAX_W 1000
8
9 // Tabla de memoización inicializada en -1 (estado no calculado)
10 int memo[MAX_N + 1][MAX_W + 1];
11
12 int max(int a, int b) {
13     return (a > b) ? a : b;
14 }
15
16 // Función recursiva con memoización
17 int mochilaTD(int i, int w, int peso[], int valor[]) {
18     // caso base
19     if (i == 0 || w == 0)
20         return 0;
21
22     // si ya calculamos este subproblema, lo devolvemos
23     if (memo[i][w] != -1)
24         return memo[i][w];
25
26     // si el objeto no cabe, lo ignoramos
27     if (peso[i - 1] > w) {
28         memo[i][w] = mochilaTD(i - 1, w, peso, valor);
29     } else {
30         // Elegimos el mejor entre no tomarlo o tomarlo
31         int no_tomar = mochilaTD(i - 1, w, peso, valor);
32         int tomar = valor[i - 1] + mochilaTD(i - 1, w - peso[i - 1], peso, valor);
33         memo[i][w] = max(no_tomar, tomar);
34     }
35
36     return memo[i][w];
37 }
38
39 int main() {
40     int valor[] = {60, 100, 120};
41     int peso[] = {10, 20, 30};
42     int W = 50;
43     int n = sizeof(valor) / sizeof(valor[0]);
44
45     // Inicializar memo en -1
46     memset(memo, -1, sizeof(memo));
47
48     printf("Valor máximo (Top-Down): %d\n", mochilaTD(n, W, peso, valor));
49
50     return 0;
51 }
52 /*Salida esperada
53 text
54 Valor máximo (Top-Down): 220*/
55
```

Valor máximo (Top-Down): 220

Process exited after 0.05098 seconds with return value 0
Presione una tecla para continuar . . .

Compilador Recursos Registro de Compilación Depuración Resultados Cerrar

Cancelar Compilación

Shorten compiler paths

Errors: 0
Warnings: 0
Output Filename: D:\escritorio_HENRY\BMA09_26_2\LCS enfoque botton up.exe
Output Size: 129.1884765625 KiB
Compilation Time: 0.16s

8. Enfoque Bottom-Up

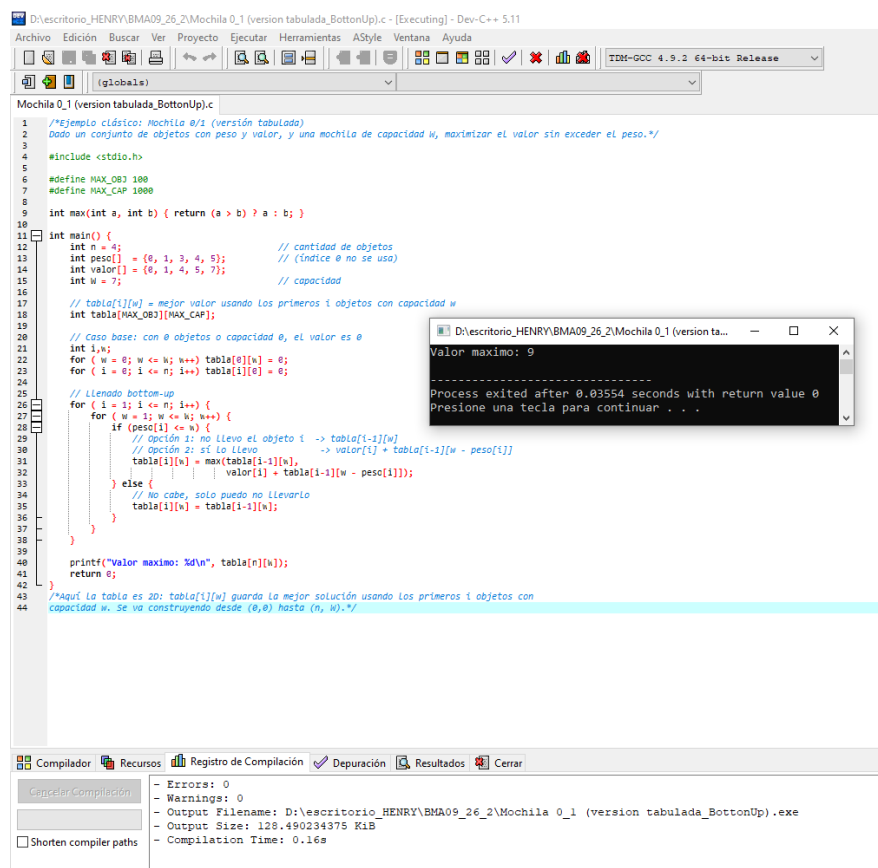
- Se resuelven primero los subproblemas más pequeños.
- Se almacenan en una tabla.
- Se construye la solución hasta llegar al problema original.

Ventaja: Generalmente más eficiente y sin recursión.

Desventaja: Puede resolver subproblemas innecesarios.

Ejemplo: Mochila 0/1 (Tabulada :enfoco Botton Up)

Dado un conjunto de objetos con peso y valor, y una mochila de capacidad W, maximizar el valor sin exceder el peso.



```
1 /*Ejemplo clásico: Mochila 0/1 (versión tabulada)
2 Dado un conjunto de objetos con peso y valor, y una mochila de capacidad W, maximizar el valor sin exceder el peso.*/
3
4 #include <stdio.h>
5
6 #define MAX_OBJ 100
7 #define MAX_CAP 1000
8
9 int max(int a, int b) { return (a > b) ? a : b; }
10
11 int main() {
12     int n = 4; // cantidad de objetos
13     int peso[] = {0, 1, 3, 4, 5}; // (índice 0 no se usa)
14     int valor[] = {0, 1, 4, 5, 7}; // capacidad
15     int W = 7;
16
17     // tabla[i][w] = mejor valor usando los primeros i objetos con capacidad w
18     int tabla[MAX_OBJ][MAX_CAP];
19
20     // Caso base: con 0 objetos o capacidad 0, el valor es 0
21     for (w = 0; w <= W; w++) tabla[0][w] = 0;
22     for (i = 0; i <= n; i++) tabla[i][0] = 0;
23
24     // Llenado bottom-up
25     for (i = 1; i <= n; i++) {
26         for (w = 1; w <= W; w++) {
27             if (peso[i] <= w) {
28                 // Opción 1: no llevo el objeto i -> tabla[i-1][w]
29                 // Opción 2: sí lo llevo -> valor[i] + tabla[i-1][w - peso[i]]
30                 tabla[i][w] = max(tabla[i-1][w], valor[i] + tabla[i-1][w - peso[i]]);
31             } else {
32                 // No cabe, solo puedo no llevarlo
33                 tabla[i][w] = tabla[i-1][w];
34             }
35         }
36     }
37
38     printf("Valor maximo: %d\n", tabla[n][W]);
39     return 0;
40 }
41
42 /*Aquí la tabla es 2D: tabla[i][w] guarda la mejor solución usando los primeros i objetos con
43 capacidad w. Se va construyendo desde (0,0) hasta (n, W).*/
```

Valor maximo: 9

Process exited after 0.03554 seconds with return value 0
Presione una tecla para continuar . . .

Compilador Recursos Registro de Compilación Depuración Resultados Cerrar

Cancelar Compilación

Shorten compiler paths

- Errors: 0
- Warnings: 0
- Output Filename: D:\escritorio_HENRY\BMA09_26_2\Mochila 0_1 (version tabulada_BottonUp).exe
- Output Size: 128.490234375 KiB
- Compilation Time: 0.16s

Ejemplo: Subsecuencia Común Más Larga (LCS)

La **Subsecuencia Común Más Larga** (Longest Common Subsequence) encuentra la secuencia de caracteres más larga que aparece **en el mismo orden** en dos cadenas, pero **no necesariamente de forma continua**.

Ejemplo:

- Cadena 1: "ABCDGH"
- Cadena 2: "AEDFHR"
- LCS: "ADH" (longitud 3)

Fíjate que las letras A, D, H aparecen en ambas cadenas en ese orden, aunque no estén juntas.

The screenshot shows a C++ IDE with a file named 'LCS enfoque bottom up.c'. The code implements the Longest Common Subsequence (LCS) algorithm using dynamic programming. It includes a function to find the maximum of two numbers, a function to calculate the LCS length, and a function to print the LCS table. The output window shows the following text:

```
=====
SUBSECUENCIA COMUN MAS LARGA (LCS)
=====
Ingrese la primera cadena: RAUL
Ingrese la segunda cadena: LUIS

--- Tabla de Programacion Dinamica ---

      L  U  I  S
R  0  0  0  0  0
A  0  0  0  0  0
U  0  0  1  1  1
L  0  1  1  1  1

La longitud de la LCS es: 1

-----
Process exited after 10.46 seconds with return value 0
Presione una tecla para continuar . . .
```

The IDE also shows the compilation results, indicating that the program compiled successfully with no errors or warnings.

9. Problemas clásicos de Programación Dinámica

Problema	Estado	Transición	Complejidad
Fibonacci	$dp[i]$	$dp[i] = dp[i-1] + dp[i-2]$	$O(n)$
Mochila 0/1	$dp[i][w]$	$\max(dp[i-1][w], val[i] + dp[i-1][w-peso[i]])$	$O(nW)$
LCS (subsecuencia común más larga)	$dp[i][j]$	$dp[i][j] = dp[i-1][j-1] + 1$ si coinciden, si no $\max(dp[i-1][j], dp[i][j-1])$	$O(nm)$
Cambio de monedas	$dp[m]$	$dp[m] = \min(dp[m - moneda] + 1)$	$O(nM)$
Camino más corto en grid	$dp[i][j]$	$dp[i][j] = \min(dp[i-1][j], dp[i][j-1]) + \text{costo}$	$O(nm)$
Distancia de edición	$dp[i][j]$	$\min(\text{insertar}, \text{eliminar}, \text{reemplazar})$	$O(nm)$